



**Virtuoso Layout Migrate
Standard Cell Migration Workshop**

gsclib090 (gpdk090) -> gsclib045 (gpdk045)

Version 6.1.3

Table of Contents

1 Overview	3
2 General Setup	3
Migratemap GUI	6
3 Creating Starting Technology File	8
Missing Rules	10
Device scaling	12
Adjust Label scaling	13
Width Dependent Spacing Adjustment	14
4 Creating Standard Cell Architecture	16
prBoundary abutment_boundary setup	16
fix NW y-position	17
fix NIMP/PIMP y-position	18
enclose layers inside prBoundary	20
Seperate M1/all into M1/drawing and M1/pin	21
add alignment locks and connectivity	23
5 Run Through Library of Cells	28
Holding Cell for Review	28
6 Schematic Migration	28

1 Overview

This Workshop covers the migration of a standard cell library from gpdk090 to gpdk045 using Virtuoso Layout Migrate. This includes the generation of a starting Virtuoso Layout Migrate specific technology file, modifying that to run smoothly, and adding standard cell architecture constraints.

This workshop does not go into the details of how to write a full setup from scratch, or all the details to options on design rules. Rather it's focus is on how to work with a pdk that has most of the design rules already in place in the dfl technology and how to supplement that to run standard cells.

2 General Setup

Virtuoso Layout Migrate requires the two technologies loaded in one session. Notice in the cds.lib both gpdk090 and gpdk045 technologies are defined.

Action 1: Start Virtuoso from command prompt

Using gsclib090 as source standard cell library, create a gsclib045 library attached to the gpdk045 technology.

Action 2: In CIW select File->New->Library...

Action 3: Enter in Name field "gsclib045"

Action 4: Click "Attach to existing technology library"

Action 5: OK the form

Action 6: Select gpdk045 to attach, and click OK

Open sample cell (AND3X2) to do general setup of the migratemap.il file.

In this case this means just a Library mapping of what source libraries should be mapped into what target libraries. If running a migration from a source library that does not have an entry in the IoLibraryMap that is generated using the GUI, then the user will get an error:

Error We are migrating library "LibraryName", but that library is not in the IoLibraryMap table.

If the source and target technologies have different layer/purpose pair usage, this also would need to be reflected in the migratemap.il file generated using the GUI.

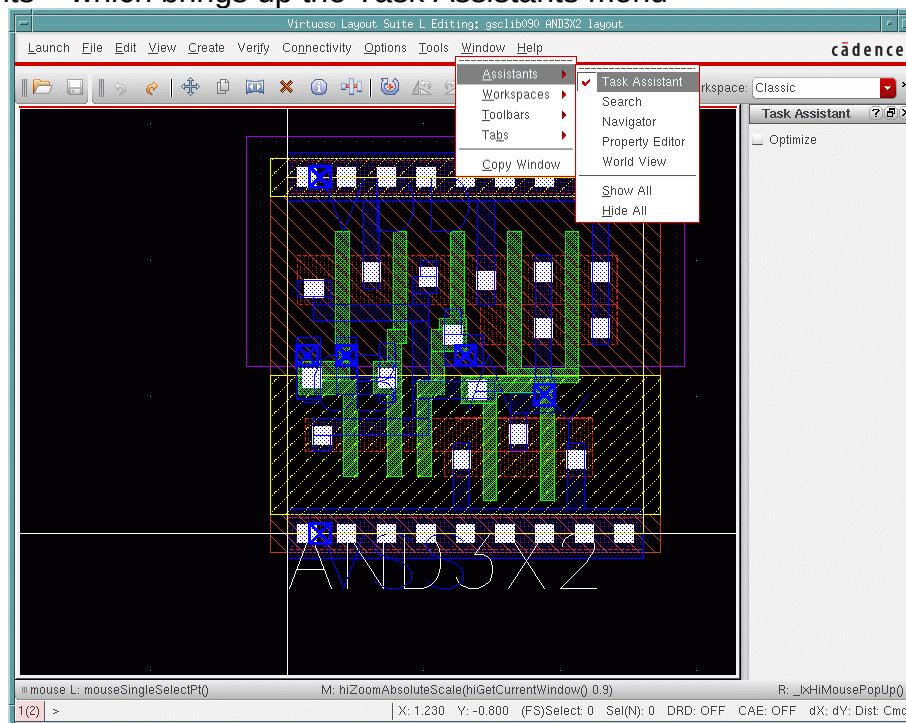
Action 7: In CIW select File->Open...”

Action 8: Open AND3X2 from the library gsclib090 in layout view



In order to get the Optimize menu in the menubar, turn on the Task Assistant and then save it to your Workspace

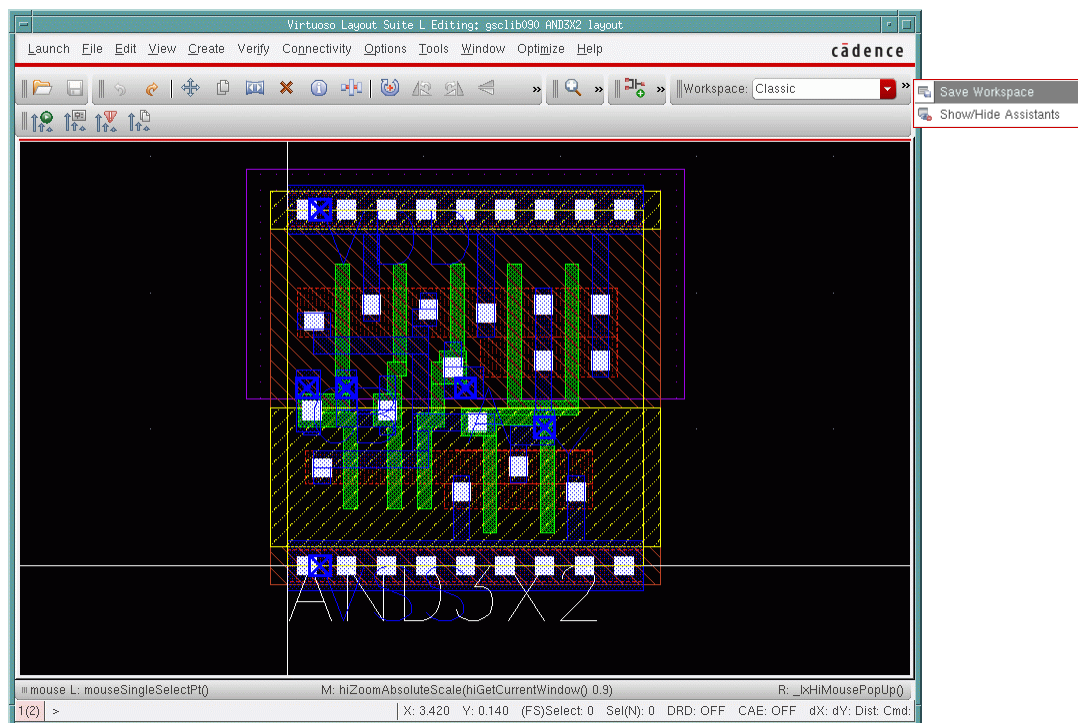
Action 9: In the AND3X2 layout view, select Window->Assistants->Task Assistants – which brings up the Task Assistants menu



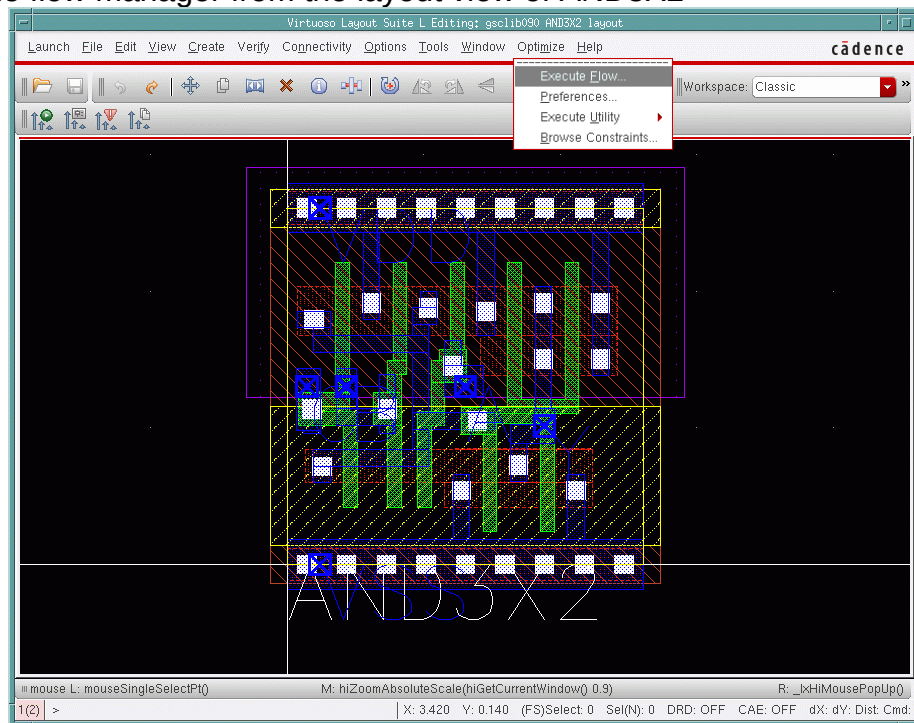
Action 10: Click the checkbox next to Optimize and close the Assistant

Action 11: Next to the Workspace access the Save Workspace menu and save

Virtuoso Layout Migrate Standard Cell Migration Workshop

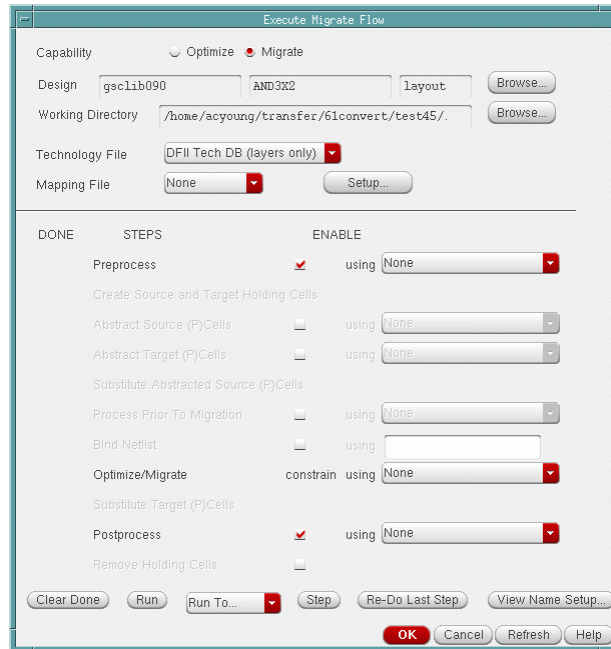


Open the flow manager from the layout view of AND3X2



Action 12: Select Optimize->Execute Flow...

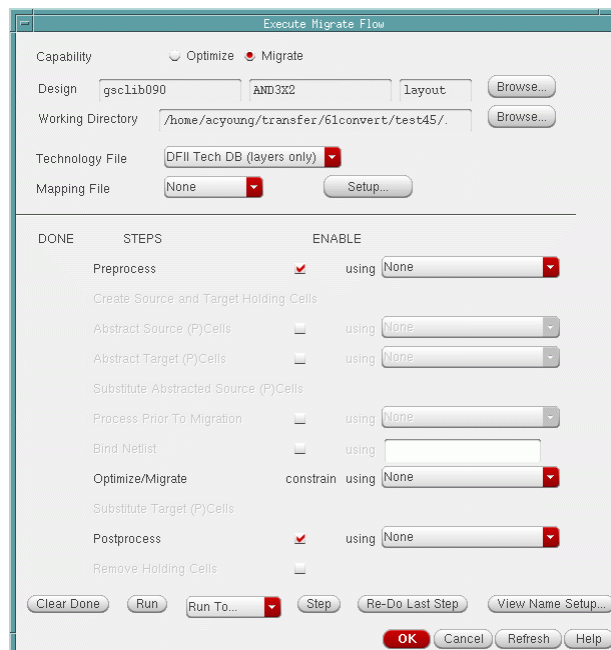
Virtuoso Layout Migrate Standard Cell Migration Workshop



Migratemap GUI

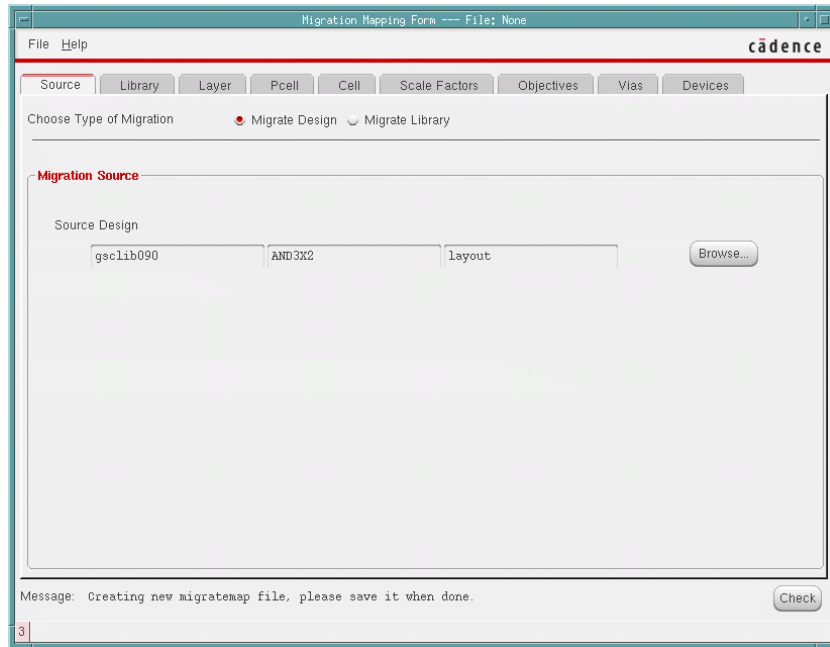
Open the migrate map GUI and map the source to target technology and design library

Action 13: In Execute Migrate Flow Form select “Setup” next to the Mapping file drop down

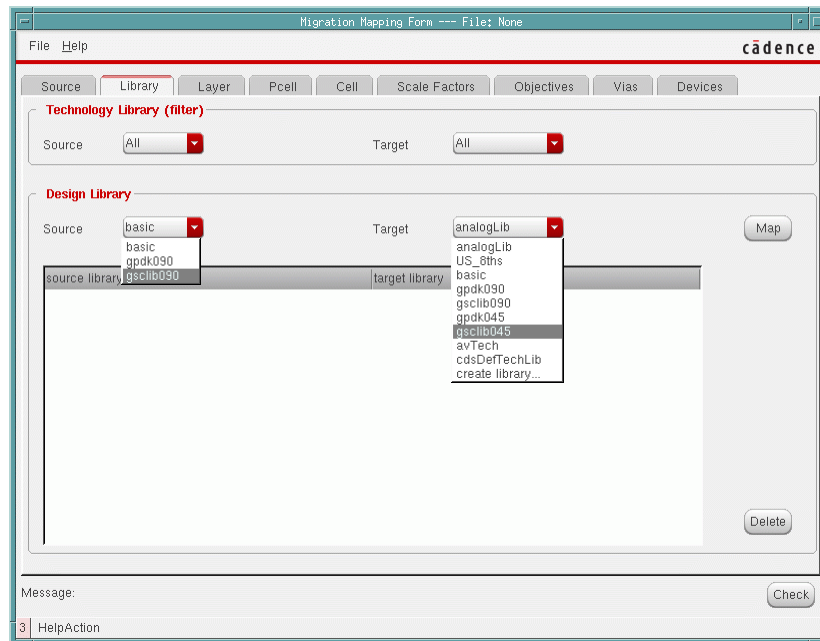


This opens the migrate mapping GUI form

Virtuoso Layout Migrate Standard Cell Migration Workshop

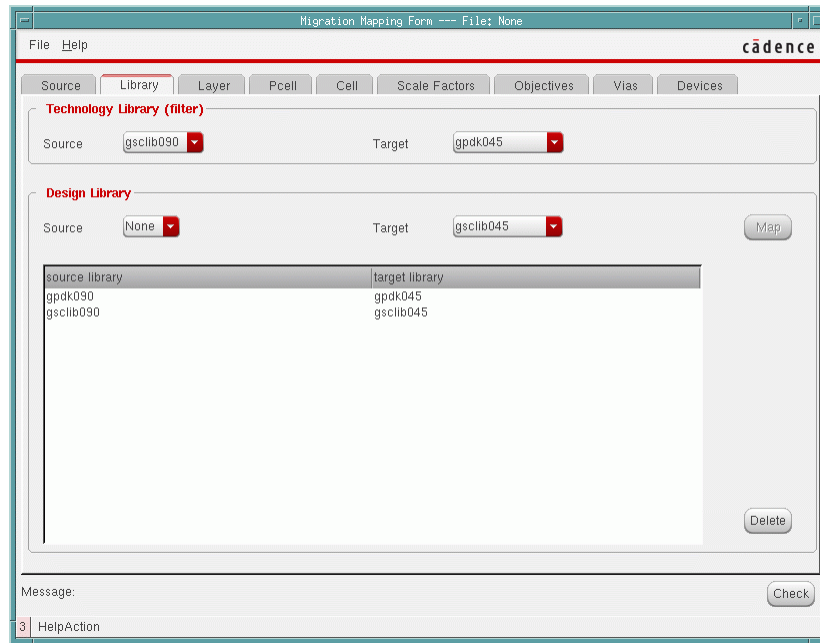


Action 14: Click on the Library tab to get the form for mapping source to target libraries



Action 15: Using the drop downs in the Design Library set the Source and Targets and click Map to store them below

gpdK090 -> gpdK045
gsclib090 -> gsclib045



Action 16: Go under the File menu to Save the mapping as migratemap.il and then close the Migration Mapping Form

3 Creating Starting Technology File

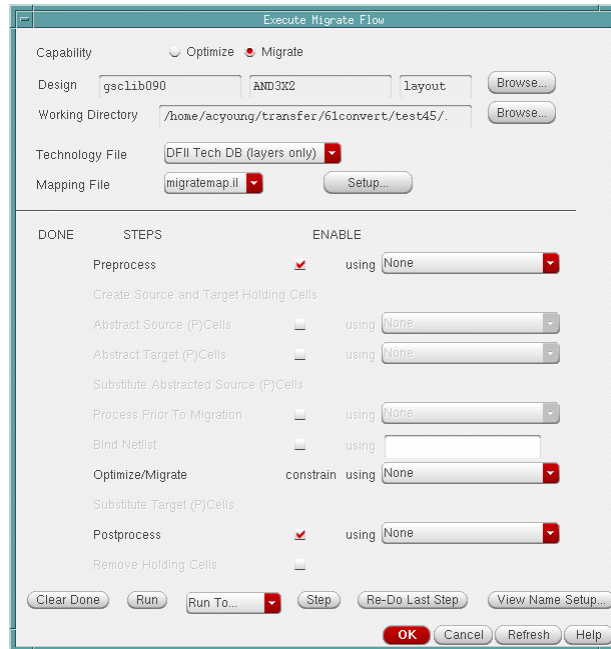
In this section, run through with "DFII Tech DB (layers only)" as techfile. Grab starting .qtt file from lodebug directory. Run a quick DRC check and add in any missing rules

The "DFII Tech DB" uses the common dfll technology database to extract information about layer usage, design rules, via handling, and device definitions that are needed in order to run Virtuoso Layout Migrate.

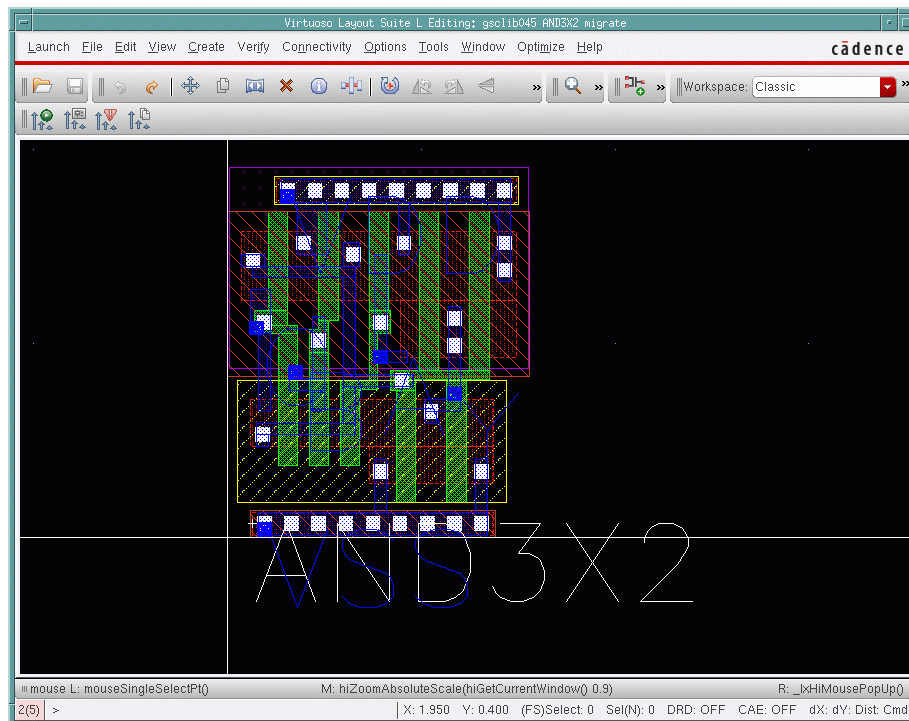
Run a quick Migration run with default settings. This will use the rules defined in the dfll technology file in the "VLMDefaultSetup" constraintGroup, which unless defined otherwise uses the "foundary" constraintGroup.

Action 1: Click Run in the Execute VLM Flow form.
Note: be sure to click "Run" and not "OK" which would only close the form.

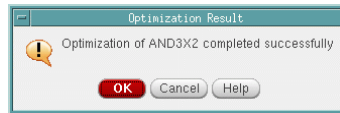
Virtuoso Layout Migrate Standard Cell Migration Workshop



This runs through the migration flow and opens up the results in a new window with the view name “migrate”



Action 2: Click OK to close the Optimization Result dialogue box



Get the text representation for the VLM technology file from the lodebug directory that gets created in the working directory.

Action 3: In command shell, copy ./lodebug/gpdk045_dfii.qtt to ./g90to45.qtt

```
% cp ./lodebug/gpdk045_dfii.qtt ./g90to45.qtt
```

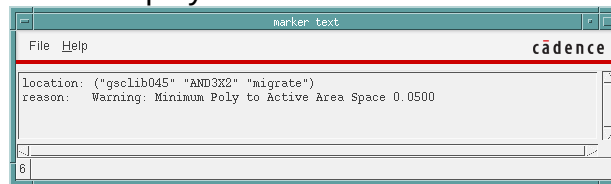
Run quick DRC. In order to access this capability we will need to switch into GXL mode. There is no need to attach any schematics as this is just layout migration we are currently working with.

Action 4: Go into GXL mode by selecting Launch->Layout GXL in the migrate view

Action 5: In migrate view select Verify->Design...

Action 6: Click OK

Browse any violations. Generally should have two types of violations, some related to metal1, and some poly to diffusion violations.



The Metal violations of space and overlap with pin will be handled later with the IOport handling. These problems arise here because all purpose types of Metal1 are being treated as the same by Virtuoso Layout Migrate in the default setup.

Missing Rules

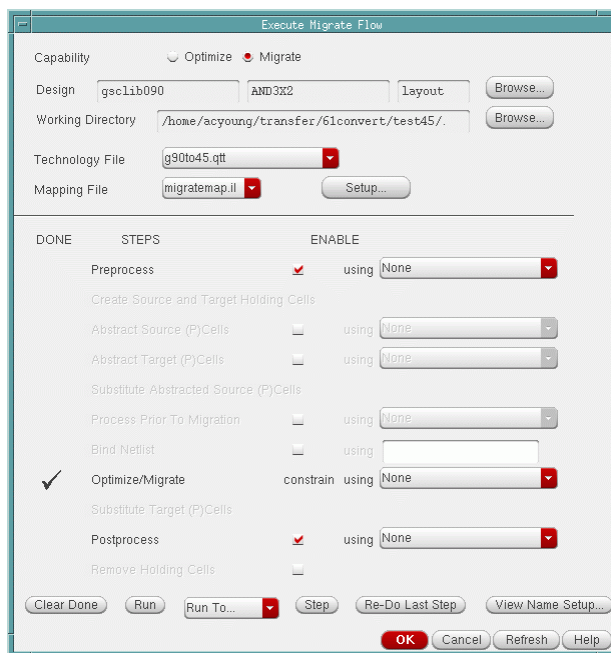
The poly to diffusion spacing violation located at x,y=(1.1,1.1) is due to missing extension rules. These missing rules need to be added to the technology file.

Action 7: Put in two rules at the bottom of the g90to45.qtt technology file

```
rules
  rule POLY.E.2 "Min POLY over OXIDE 0.1" extend POLY over OXIDE by 0.1
  rule POLY.E.3 "Min diff over POLY 0.1" extend OXIDE over POLY by 0.1
end
```

Rerun the migration flow again using this modified technology file to compare results

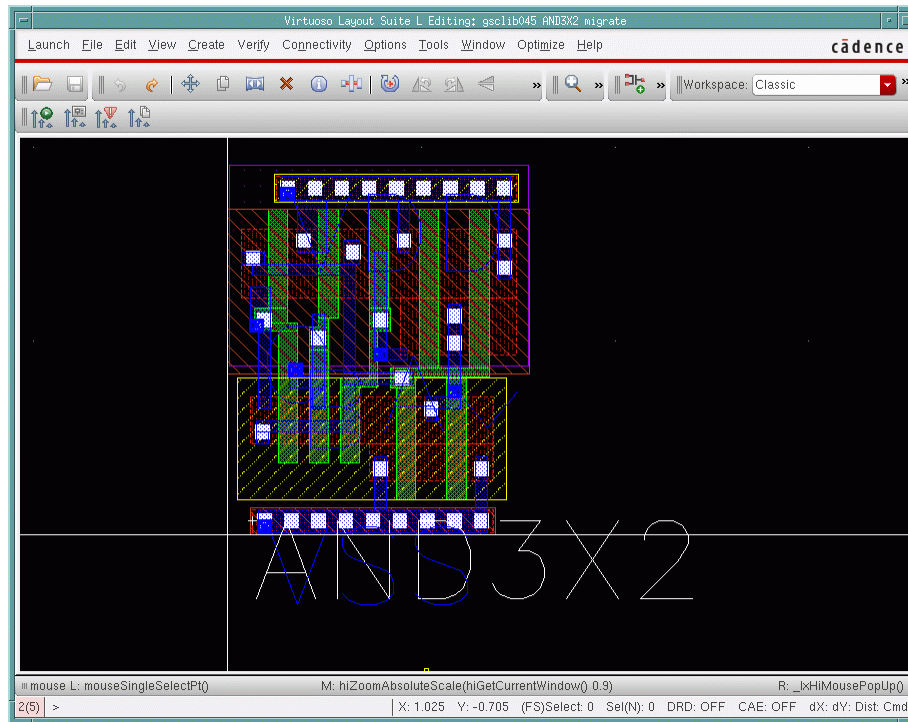
Action 8: Change the Technology File drop down to use g90to45.qtt
This should require clicking “Refresh” in the bottom right corner of the form first in order to recollect the available files, including “Technology Files” (all files ending in .qtt)



Action 9: Click “Clear Done” and then “Run” in the Execute Migrate Flow menu
Make sure that the design was run still from gscLib090/AND3x2/Layout as the source design as otherwise you may get an error at this point stating
Error We are migrating library "gscLib045", but that library is not in the loLibraryMap table.

This occurs when the Execute Migrate Flow menu is initialized again from the window you select “Optimize->Execute Flow..”

The resulting migrate view should no longer have POLY to OXIDE violations



Device scaling

Devices were maintained at the same width and length. We want to adjust the device scaling technology file to 0.45 length scale and 0.6 width scale. These values are somewhat arbitrary, but in our example, they are based upon desiring minimum device lengths (migrating from 0.1 in source to 0.045 in target) along with eventually the height of the cell being scaled down by around 0.6

Also we will use a scale parameter to make it easier to adjust if further changes are necessary

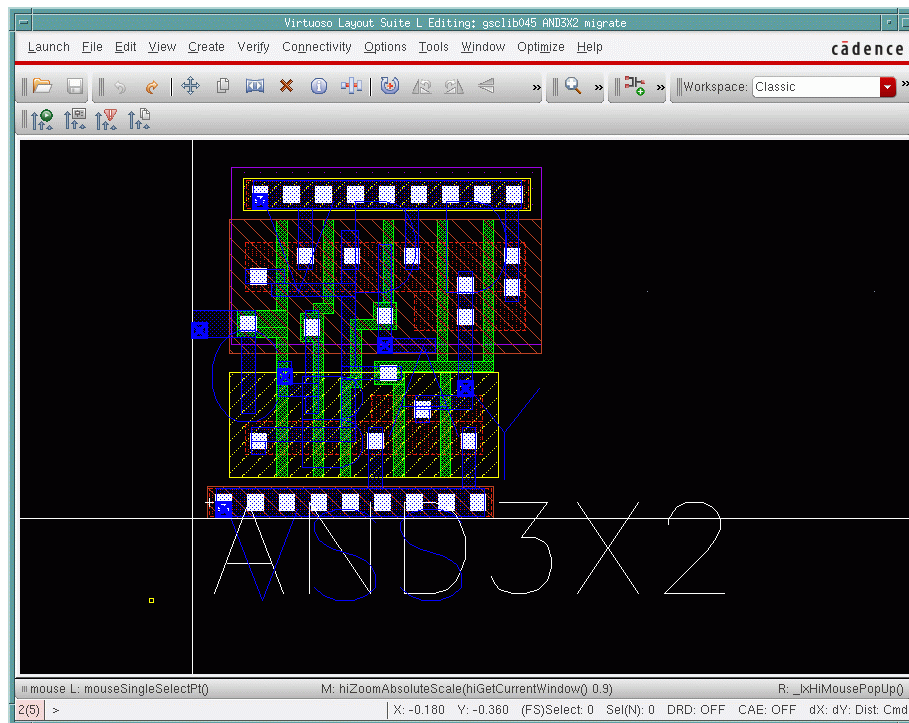
Action 10: At the top of the g90to65.qtt file, change the “parameters” section to “scale_parameters” with two parameters:

```
scale_parameters
deviceLscale = 0.45
deviceWscale = 0.6
end
```

Action 11: In the g90to45.qtt change the first two devices lines (about line 351 and 352) to:

```
MOSFET NMOS11 ngate11 OXIDE POLY substrate length scale deviceLscale
width scale deviceWscale
MOSFET PMOS11 pgate11 OXIDE POLY NWELL length scale deviceLscale
width scale deviceWscale
```

Action 12: Rerun migration again by clicking “Clear Done” and then “Run” in the Execute Migrate Flow form again and then OK the Optimize Results dialogue box



Adjust Label scaling

All labels in the design are maintained at the same size as the source. In order to keep the labels in general at the same scale as the design, create another parameter, recognize Metal1/label as a layer, and adjust all the label scaling

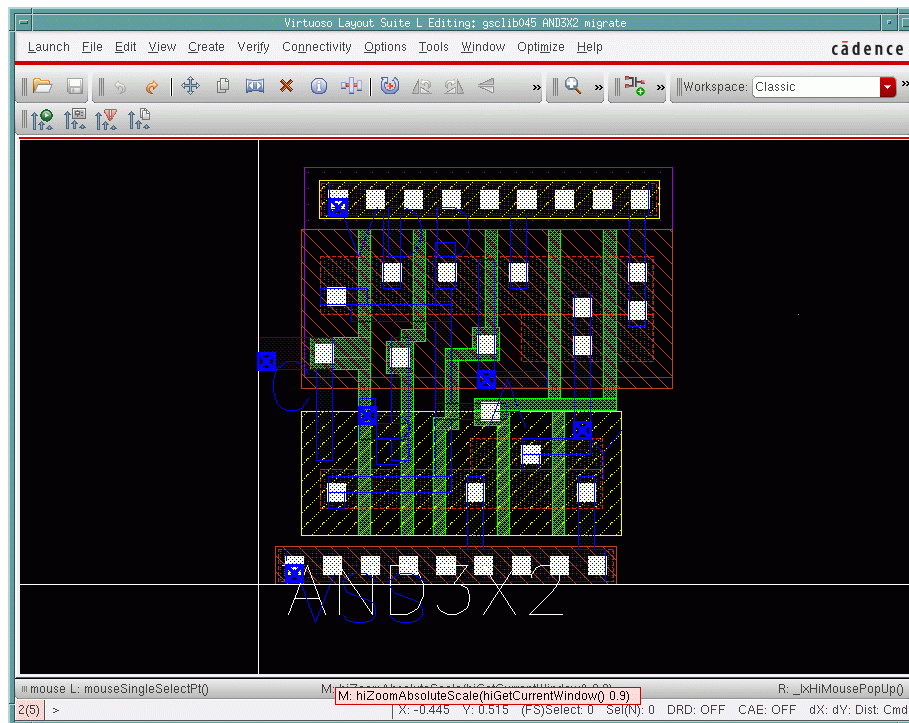
Action 13: Create labelscale parameter in technology file parameters
 labelscale = 0.45
 ...

Action 14: Add layer M1L after METAL1 around line 25
 METAL1 alias Metal1/drawing
 M1L alias Metal1/label

Action 15: Add the new layer M1L to the Metal1 plane around line 147
 METAL1 (CONT METAL1 M1L VIA1 prBoundary snapBoundary CAPMETAL)

Action 16: In the labels section (~line 153) add two lines with scaling labels
 M1L text_for METAL1 scale labelscale
 TEXT text_for METAL1 or prBoundary scale labelscale

Action 17: Rerun migration again by clicking “Clear Done” and then “Run” in the Execute Migrate Flow form again and then OK the Optimize Results dialogue box
 Notice the label size should be scaled down by 0.45 from the source label size



Width Dependent Spacing Adjustment

This section may not be necessary. The width depend spacing rules need some work on what gets generated from the dfl technology into .qtf technology file. (CCR665062 has been filed regarding this)

Action 18: Convert the Metal1 width dependent spacing rules to simplified groups.

```
rule foundry_METAL1_gt_0.0005_minSpacing
"foundry_METAL1_gt_0.0005_minSpacing is 0.06"
space METAL1 by 0.06

rule foundry_METAL1_gt_0.1005_minSpacing_common-run_0.3205
"foundry_METAL1_gt_0.1005_minSpacing is 0.06 common-run >= 0.3205 is 0.1"
space METAL1_gt_0.1005 by 0.1

rule foundry_METAL1_gt_0.7505_minSpacing_common-run_0.7505
"foundry_METAL1_gt_0.7505_minSpacing is 0.06 common-run >= 0.3205 is 0.1
common-run >= 0.7505 is 0.25"
space METAL1_gt_0.7505 by 0.25

rule foundry_METAL1_gt_1.5005_minSpacing_common-run_1.5005
"foundry_METAL1_gt_1.5005_minSpacing is 0.06 common-run >= 0.3205 is 0.1
common-run >= 0.7505 is 0.25 common-run >= 1.5005 is 0.45"
space METAL1_gt_1.5005 by 0.45

rule foundry_METAL1_gt_2.5005_minSpacing_common-run_2.5005
"foundry_METAL1_gt_2.5005_minSpacing is 0.06 common-run >= 0.3205 is 0.1
```

```
common-run >= 0.7505 is 0.25 common-run >= 1.5005 is 0.45 common-run >=
2.5005 is 0.75"
space METAL1_gt_2.5005 by 0.75
```

```
rule foundry_METAL1_gt_3.5005_minSpacing_common-run_3.5005
"foundry_METAL1_gt_3.5005_minSpacing is 0.06 common-run >= 0.3205 is 0.1
common-run >= 0.7505 is 0.25 common-run >= 1.5005 is 0.45 common-run >=
2.5005 is 0.75 common-run >= 3.5005 is 1.25"
space METAL1_gt_3.5005 by 1.25
```

Action 19: Optional copy the saved .qtt file (g90to45.qtt.final) over your existing g90to45.qtt if you had any difficulties with the previous actions in modifying the g90to45.qtt file.

4 Creating Standard Cell Architecture

In this section, you will create a standard cell architecture. This includes fixing the abutment boundary so that cells can be abuted next to each other and meet all design rules and match up locations. These fixed locations can all be considered arbitrary in their values, but in this exercise we will use values that are for the most part scaled down from their equivalent in the gpd090.

All the standard cell specific rules and settings will be put in a separate file that is sourced into the g90to45.qtt technology file. Put all new entries into a file named "extrarules"

Action 1: Create new file extrarules, and add in to the bottom of g90to45.qtt
`source ./extrarules`

prBoundary abutment_boundary setup

Fix cell height (can show calculations of estimated number of tracks from source to target scaling)

The source cell is 2.61um high, which we can measure out to be 9 grids at 0.29 each grid. In the target technology, the grid can be set to be 0.18

Action 2: In extrarules, add in a parameters section first

```
parameters
  xiogrid = 0.18
  yiogrid = 0.18
  Numgrids = 9
  CellHeight = Numgrids * yiogrid
end
```

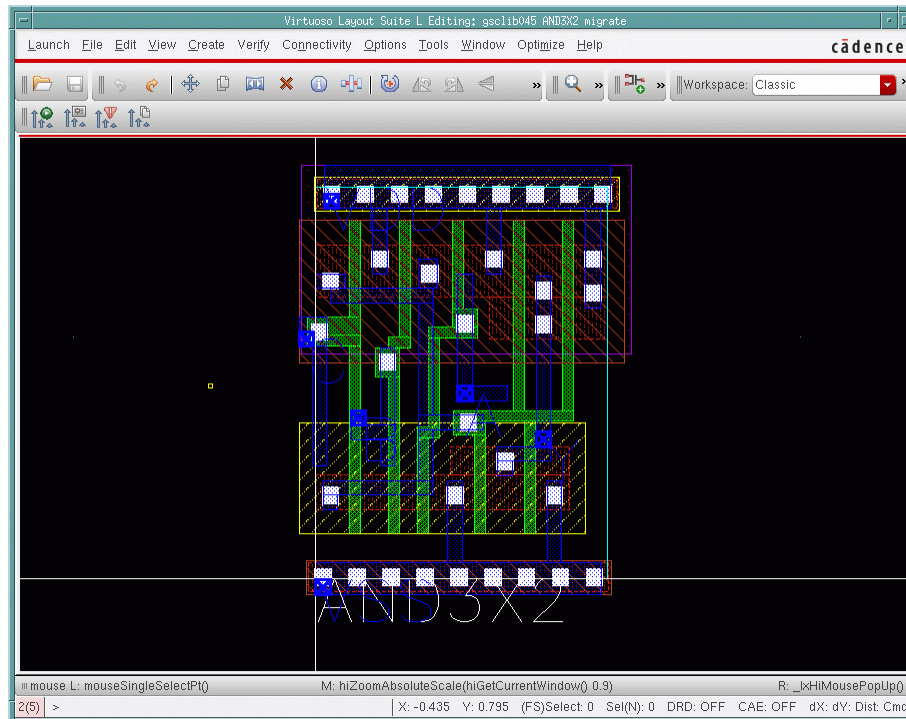
Action 3: Then add in a new section for fixing the cell height

```
abutment_boundaries
  prBoundary origin 0,0 height CellHeight iogrid xiogrid yiogrid
end
```

Action 4: Create two new rules for minimum size and keeping objects inside the prBoundary. The 0.1 value is an arbitrary small number greater than 0 for setting some minimum width on prBoundary just so that it can not be zero width.

```
rules
  rule prBoundary.W "Min prBoundary width" widen prBoundary by 0.1
  rule prBoundary.E "Min prBoundary width" extend prBoundary over OXIDE by 0
end
```

Action 5: Rerun migration again by clicking "Clear Done" and then "Run" in the Execute Migrate Flow form again and then OK the Optimize Results dialogue box



Fix NWELL y-position

In order for cells to be able to abut together, the NWELL needs to be at the same position on every cell. To do this requires having fixed values from the top and bottom of the fixed prBoundary positions.

Action 6: Add to the parameters section of the extrarules technology file two parameters.

The value for these parameters should reflect something similar to the source design where the NWELL position was $y=1.22\mu\text{m}$ with a cell height of $2.61\mu\text{m}$ – with the new cell height being 1.62 , then the NWELL position could be calculated to be about $1.22 / 2.61 * 1.62 = 0.757\mu\text{m}$.

parameters

NWfromBot = 0.78

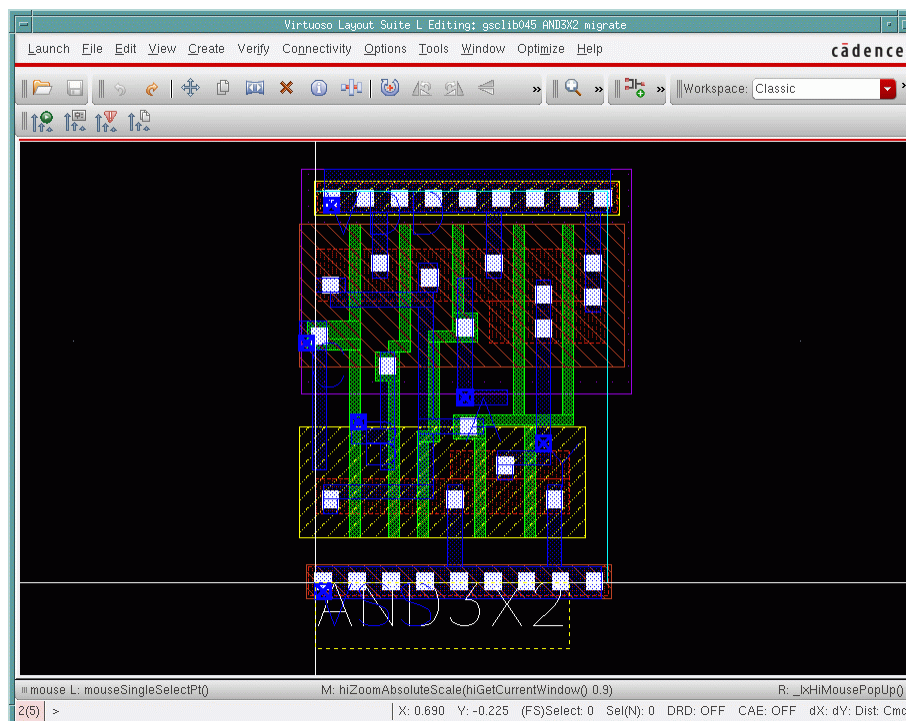
NWfromTop = CellHeight – NWfromBot

Action 7: Add in two rules to the extrarules

rule prby.NW.Top "NWEll in from Top" extend NWEll over ~prBoundary by NWfromTop

rule prby.NW.Bot "NWEll up from Bottom" extend prBoundary over NWEll by NWfromBot

Action 8: Rerun migration again by clicking “Clear Done” and then “Run” in the Execute Migrate Flow form again and then OK the Optimize Results dialogue box



Fix NIMP/PIMP y-position

Similar to NWELL needing to be fixed in location, the NIMP and PIMP also need to be fixed at the boundary. However, it requires a few more points to fix the NIMP and PIMP locations as they have multiple geometries and the fixed position should only be applied at the boundary, and potentially not in the interior of the cell where the NIMP and PIMP structures may need to jog up and down. Also maintain complete coverage of the whole cell with either NIMP or PIMP by adding in a connectivity section.

Action 9: Add to the parameters section of the extrarules technology file two parameters.

Once again, very similar to the NWELL fixed position, the position of the NIMP to PIMP boundary along the edges may be arbitrarily chosen, but it should fit all possible cells. In order to do this something similar to the source technology should be used again where it is 1.02um, scaled by the same as the cell height, results in something close to 0.633um.

The 0.16um subtracted from the NPprbySpace is based on the minimum contact enclosure at the top and bottom boundary of the cell (half a contact width + min enclosure)

parameters

PPprbySpace = 0.68

NPprbySpace = CellHeight - PPprbySpace - 0.16

Action 10: In order to limit the place where the rules will be applied, we need a few functions added to the extrarules. The value 0.01 is a small number (2 manufacture grids in this case) to find the area just along the left and right edges of the prBoundary.

```
functions
  prbyShrinkX = prBoundary - 0.01 xonly
  prbyEdge = prBoundary & ~ prbyShrinkX
end
```

Action 11: Add several rules which fix the total NIMP and PIMP area. The value of 0.06 is another arbitrary value used for a minimum extension out to the side. Following the same method as the gpdk090 of using the value that is half the width of NIMP and PIMP.

```
rule prby.NIMP.E "Extend NIMP over prby" extend NIMP over prBoundary by
0.06
rule prby.PIMP.E "Extend PIMP over prby" extend PIMP over prBoundary by
0.06

rule prby.NIMP.S "Space NIMP at boundary" space NIMP over prbyEdge by
NPprbySpace
rule prby.PIMP.S "Space PIMP at boundary" space PIMP over prbyEdge by
PPprbySpace
```

Action 12: Add to the extrarules file a connections section of the technology file. What this does is indicate to Virtuoso Layout Migrate that when in the source NIMP and PIMP touch, to maintain that same touching relationship. Without it, you should be able to see that they separate in the migrated results.

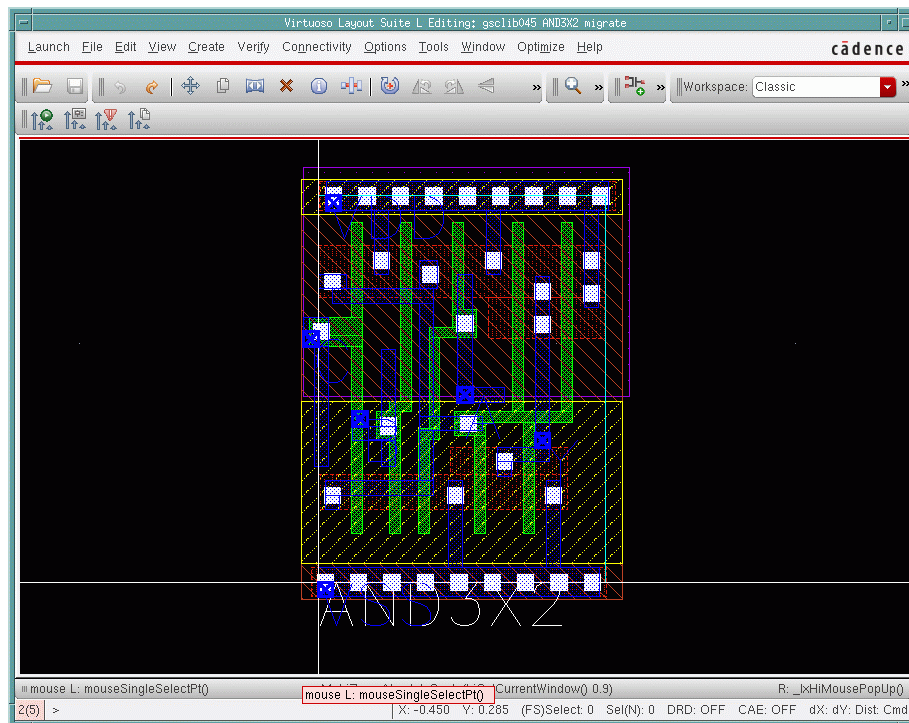
```
connections
  NIMP to PIMP overlap
end
```

When we want to maintain this touching relationship between NIMP and PIMP, there are some cells where there is butted diffusion, where ndiffusion is directly connected to pdiffusion. In order to allow for this, there are a number of rules that need to be modified in the main g90to45.qtt to allow for them to remain abutted. They should be around lines 601,625,628,1334, and named
foundry_ndiff_pdiff_minSpacing
foundry_NIMP_ptap_minSpacing
foundry_PIMP_ntap_minSpacing
foundry_NIMP_pdiff_NWELL_minSpacingOver

Action 13: Modify each of those rule lines by adding in “unless butted” before the by value statement in the g90to45.qtt:

```
space ndiff from pdiff unless butted by 0.1
space NIMP from ptap unless butted by 0.02
space PIMP from ntap unless butted by 0.02
space NIMP from pdiff over NWELL unless butted by 0.08
```

Action 14: Rerun migration again by clicking “Clear Done” and then “Run” in the Execute Migrate Flow form again and then OK the Optimize Results dialogue box



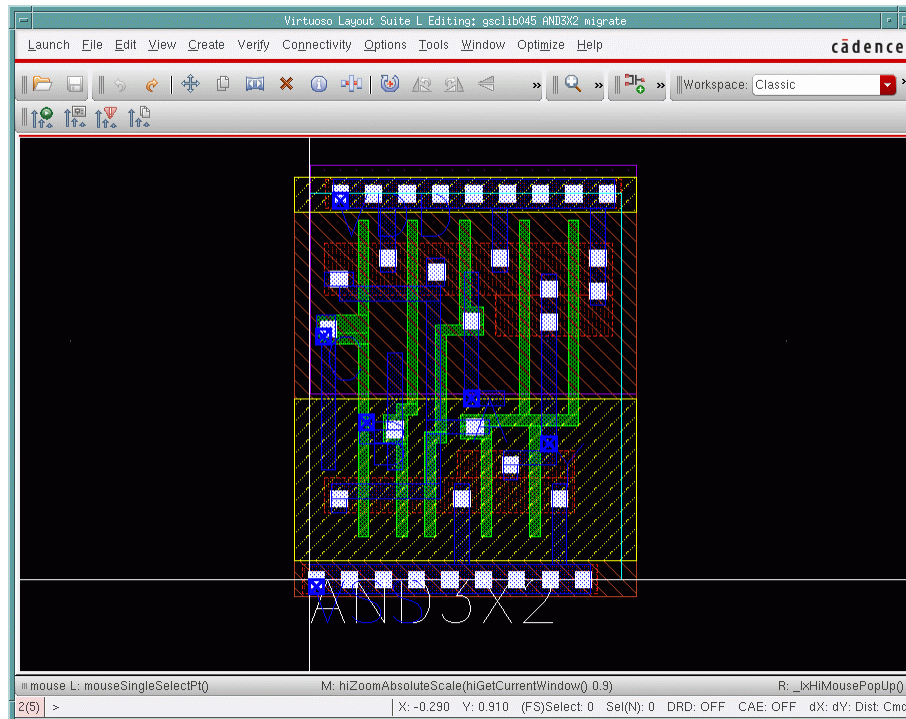
Enclose Layers Inside prBoundary

Looking at previous results you'll see that there are several layers that extend outside the prBoundary that should keep inside. In order for two cells to abut properly, all the interior layers should be enclosed by the prBoundary by half the spacing value

Action 15: Add to the extrarules file several more rules, one for OXIDE, POLY, and METAL1 to be enclosed with prBoundary, each of which is half the minimum spacing value for that layer.

```
rule OXIDE.prby.E "enclose OXIDE with prby" enclose OXIDE with prBoundary
unless coincident by 0.04
rule POLY.prby.E "enclose POLY with prby" enclose POLY with prBoundary
by 0.03
rule METAL1.prby.E "M1P inside prby" enclose METAL1 with prBoundary
unless coincident by 0.03
```

Action 16: Rerun migration again by clicking "Clear Done" and then "Run" in the Execute Migrate Flow form again and then OK the Optimize Results dialogue box



Add IOport handling

The Metal1/pin layer (M1P) should be treated as an IOport, and so needs to be added to the g90to45.qtt technology for special handling different from Metal1/all. This includes adding it for layer recognition, belonging to a plane, adjusting M1L to keep over Metal1/pin, and recognizing it as a via type all in the g90to45.qtt technology file. For further handling specific to standard cell architectures, the M1P needs to be recognized in the abutment boundaries section as the IOport and have some rules added for basic handling in the extrarules file.

Action 17: First add in the M1P layer below the existing METAL1 layers entry layers

```
...
METAL1 alias Metal1/drawing
M1P alias Metal1/pin
...
```

Action 18: Add that to the METAL1 plane (~line 148)
METAL1 (CONT METAL1 M1P M1L VIA1 prBoundary snapBoundary
CAPMETAL)

Action 19: Adjust the existing entry for M1L (~line 153)
M1L text_for M1P or METAL1 scale labelscale

The M1P should reserve space for a VIA1 for connections, so the size follows the VIA1 rule (0.07 min width). Just like in the source it's oversized by 0.005 on both sides so that the Metal1 enclosure can be 0 on the sides.

Action 20: Add M1P into the vias section (~line 343)

M1P M1P METAL1 size 0.08

Action 21: In extrarules, add to the abutment_boundaries section

prBoundary origin 0,0 height CellHeight iogrid xiogrid yiogrid
ports M1P leftright halfgrid topbottom halfgrid

Action 22: Finally add into the extrarules file several more rules for the M1P handling. These numbers all following the VIA1 rules where instead M1P is sized up by 0.005 on each side.

rule M1P.W "M1P minWidth is 0.08" widen M1P by 0.08
rule M1P.S "M1P space xiogrid - 0.08 (0.1)" space M1P by xiogrid - 0.08 corner
manhattan
rule M1P.E "M1P enclose by METAL1" enclose M1P with METAL1 by 0 end-of-
line 0.025

Action 23: Optionally create a file constraints.qts and do soft freeze on M1P

The word "qtrec" is a keyword indicating it's a script going to the original executable named qtrec.exe

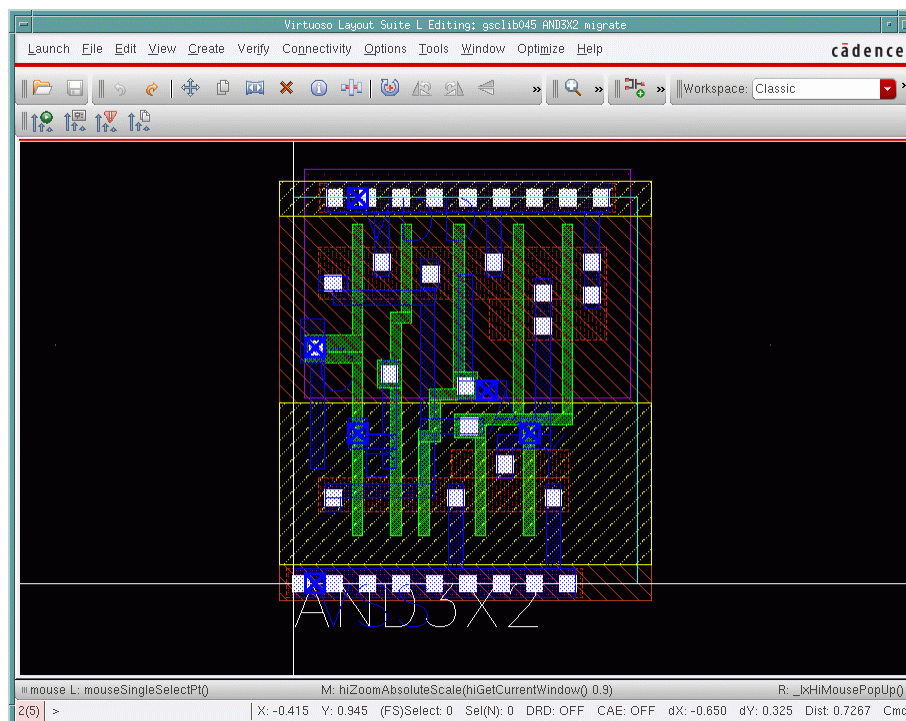
When we soft freeze the M1P, we should also indicate that the overall abutment boundary box should not follow the same scaling factor, and instead should be something much smaller

qtrec
soft freeze * layers M1P scale 0.18/0.29
soft freeze * layers prBoundary scale 0.1

Action 24: Activate the constraints.qts file by setting it at the optimize line of the Execute Migration Flow menu

Action 25: Rerun migration again by clicking "Clear Done" and then "Run" in the Execute Migrate Flow form again and then OK the Optimize Results dialogue box

Virtuoso Layout Migrate Standard Cell Migration Workshop



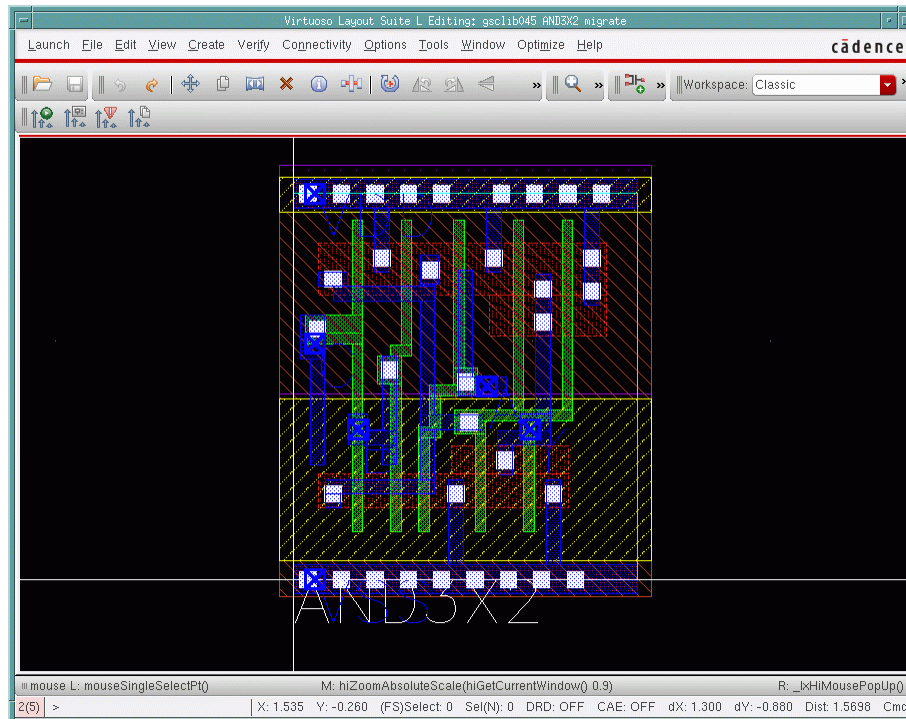
Alignment Locks at boundary

Add alignment locks for METAL1 and OXIDE so that they remain locked to the prBoundary at the left and right borders so that the tab regions are continuous

Action 26: Add to the extrarules file a new section

```
alignment_locks
prBoundary OXIDE
prBoundary METAL1
end
```

Action 27: Rerun migration again by clicking “Clear Done” and then “Run” in the Execute Migrate Flow form again and then OK the Optimize Results dialogue box



merge_vias and split_vias

Finally now you can see that the contacts along the top and bottom boundaries do not fully span the regions. Will add in two scripts preprocess and postprocess.qts in order to get exact placement of contacts on boundary. The preprocess file can also be used to merge other geometries for easier handling during the migration process

Action 28: Create a new file preprocess.qts and add

```
qtrek
function srcdrn = ( sd_ndiff | sd_pdiff ) & ~ POLY
merge_vias nopolygons * CONT srcdrn 0.2
function taps = ntap | ptap
merge_vias nopolygons * CONT taps 0.2
```

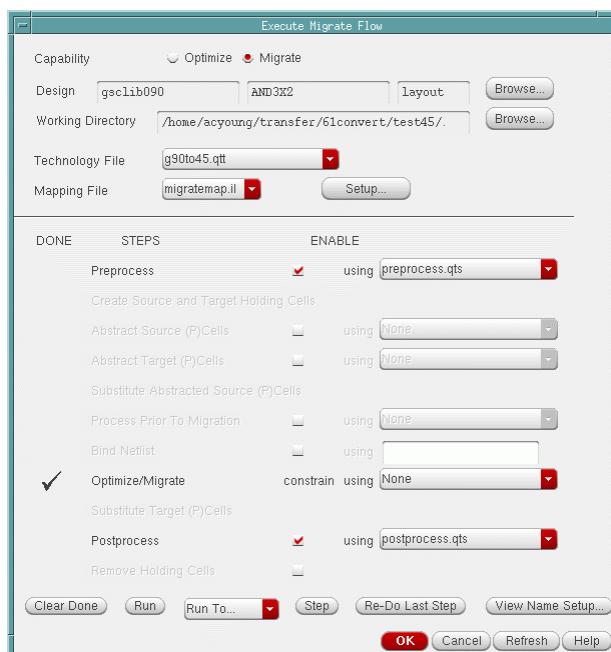
Action 29: In order to get the contacts to fill all available area, set an objective grow force on CONT (~line 307 of g90to45.qtt)

```
objectives
CONT growforce 6
end
```

Action 30: Create a new file postprocess.qts and add

```
qtrek
split_vias * CONT ntap spacing 0.12 maximum 0.12
split_vias * CONT ptap spacing 0.12 maximum 0.12
split_vias * CONT CONT spacing 0.06
```


Action 31: Activate the preprocess and postprocess scripts by selecting them from the dropdowns in the Execute Migrate Flow menu

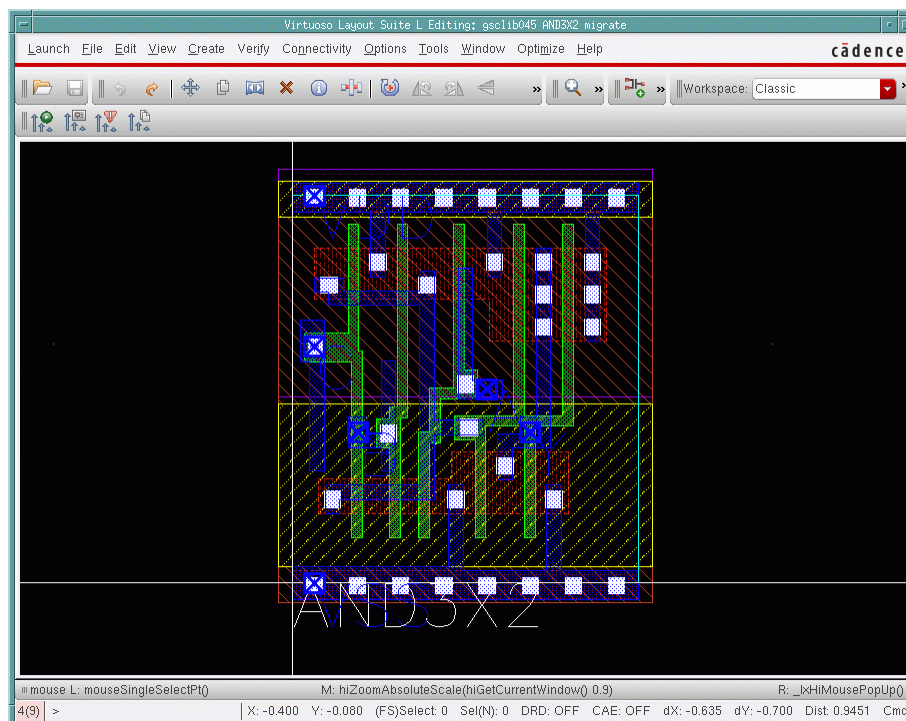


Action 32: Add to the preprocess.qts a few more lines of scripting that help clean up the results

```
merge_elements * NWELL NWELL
merge_elements * POLY POLY
merge_elements * OXIDE OXIDE
merge_elements * METAL1 METAL1
```

```
shrink * CONT CONT 0.01
shrink *FILL* M1P M1P 0.02
```

Action 33: Rerun migration again by clicking “Clear Done” and then “Run” in the Execute Migrate Flow form again and then OK the Optimize Results dialogue box



Center Contacts at Top/Bottom Boundary

If you look very closely, you will also still see the contacts are not exactly centered over the prBoundary like in the source layout. In order to force the contacts to be centered, two final additional rules need to be added.

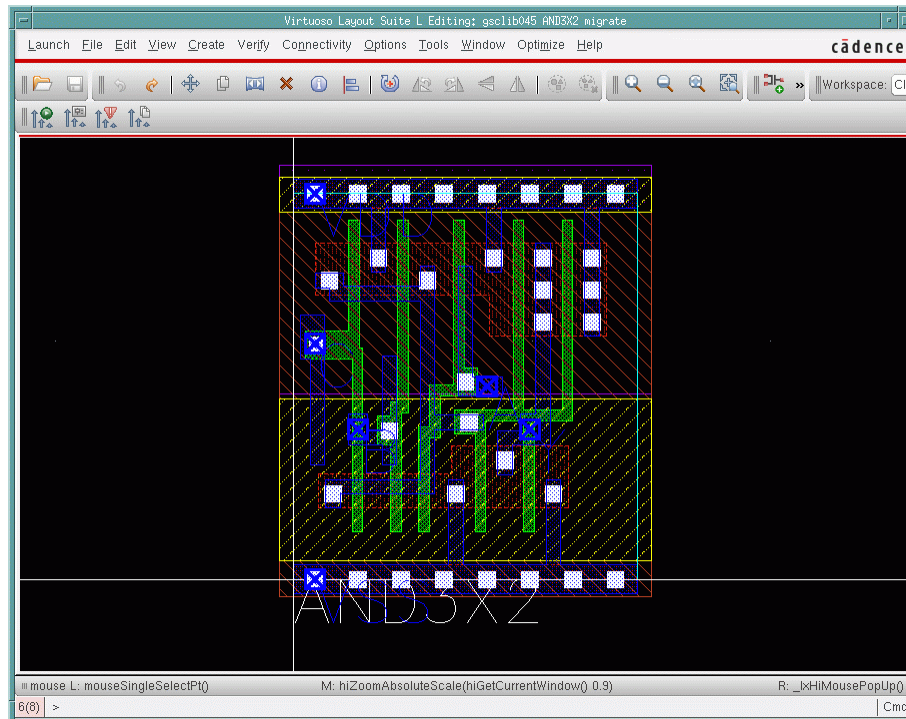
Action 34: Finally add into the extrarules file

Rule CO.PRBY.IN "CONT in PRBY" extend CONT over ~prBoundary by 0.03

Rule CO.PRBY.OUT "CONT out PRBY" extend CONT over prBoundary by 0.03

Action 35: Rerun migration again by clicking "Clear Done" and then "Run" in the Execute Migrate Flow form again and then OK the Optimize Results dialogue box

Virtuoso Layout Migrate Standard Cell Migration Workshop



5 Run Through Library of Cells

How to run through a library. Two scripts - one to run, one to make holding cell

Action 1: In CIW: load “./allcells.il”

Action 2: Create list of all cells. Command prompt
% ls gsclib090 > celllist

Action 3: Run through all cells in a list: CIW: OptCellsInFile(“celllist” “gsclib090”)

Holding Cell for Review

Create holding cell view and use script to place all cells

Action 4: load “./makeholding.il”

Action 5: CIW: makeholding “celllist” “gsclib045”

Action 6: Run DRC – Assura -> Run...

6 Schematic Migration

Schematic migration. Do a single migration first with setup of the migratemap.il file

- mos mapping

Use script to place all cells, run schematic migration on them all.

Action 1: Put in mapping for nmos/pmos symbols into migratemap.il file

```
loParameterMap["mosmap"] = list(
    ;list(targetParam function of source parameter)
    list("w" "cond( (stringp(w) cdfFormatFloatString(sprintf(nil
\\\"%g\\\",round(cdfParseFloatString(w) * 1e6/0.005 * .6)*1e-6*0.005),\\\"n\\\")) (t
round(w*1e6/0.005 * .6)*1e-6*0.005)))")
)

loPcellMap = list(
    ;list(sourcePcell inst parent targetPcell parameterMap target-rotation post-
callback)
    list("gpd090/pmos1v/symbol" "*" "*" "gpd045/pmos1v/symbol" "mosmap"
"R0" "None")
    list("gpd090/nmos1v/symbol" "*" "*" "gpd045/nmos1v/symbol" "mosmap"
"R0" "None")
)
```

Action 2: Run schematic migration